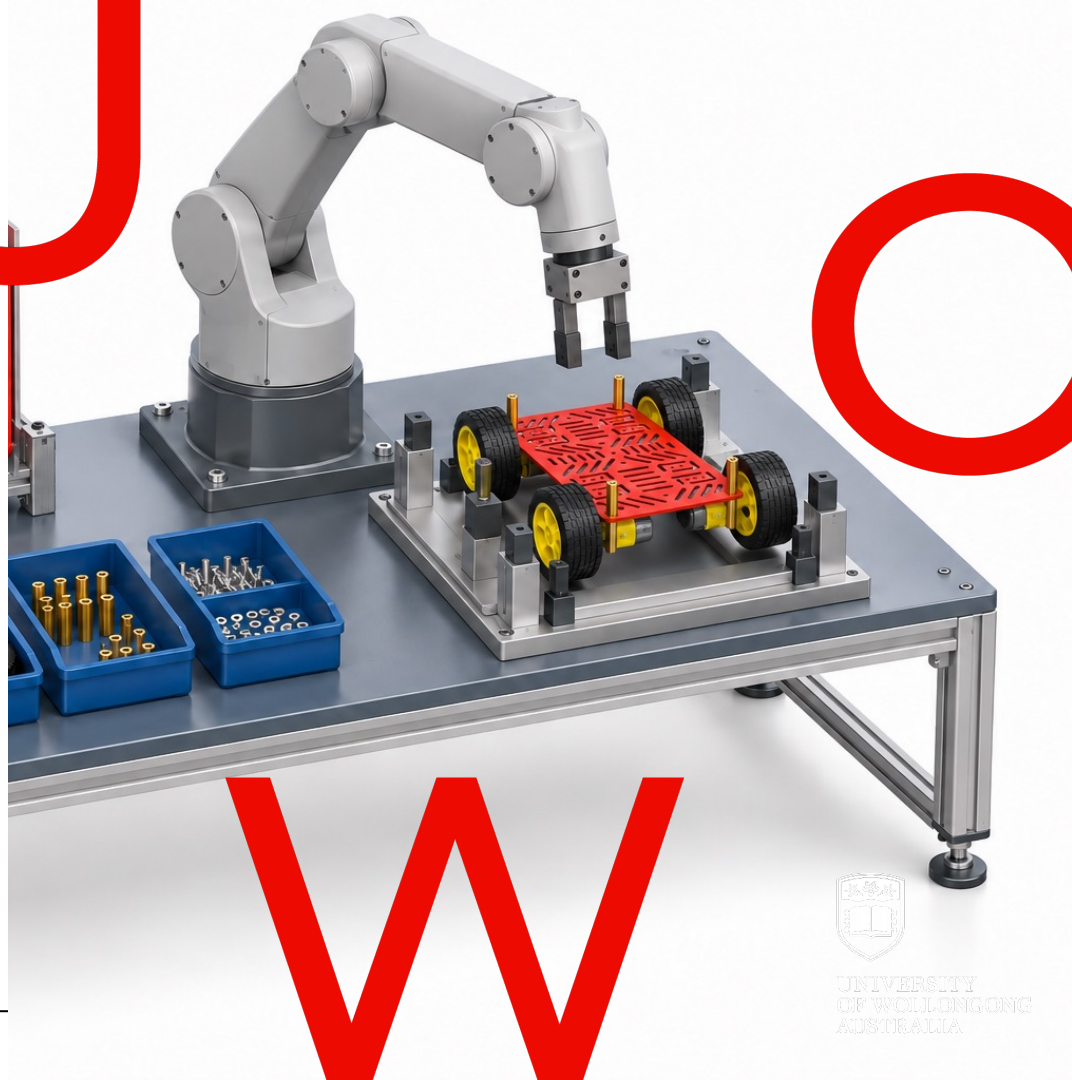
Robotics and Flexible Automation

Lecture 8
Machine Vision and Calibration



Prof. Zengxi (Stephen) Pan

BJTU 2026

Machine vision in a robotic system

CORE

how image data becomes a pose that the robot can use

	ROBOT QUESTION	VISION METHOD	ROBOT-READY RESULT
1	**Where is the correct part?**	Image data and feature extraction	**Identity and 2D location**
2	**How is it oriented?**	Camera model and geometric transformations	**3D pose and orientation**
3	**Is it within tolerance?**	Calibration and metric measurement	**Size and position error**
4	**Where should the tool move?**	Transform the target into the robot base frame	**Robot motion target**

IMAGE COMPRESSION supports storage and transfer across the workflow

Pixels become useful to the robot when calibration connects image measurements to the robot base frame.

Application: turbine blade repair

EXAMPLE

cavitation damage on hydro turbine runners, repaired by robot



damaged runner, and the cavitation pitting on a blade

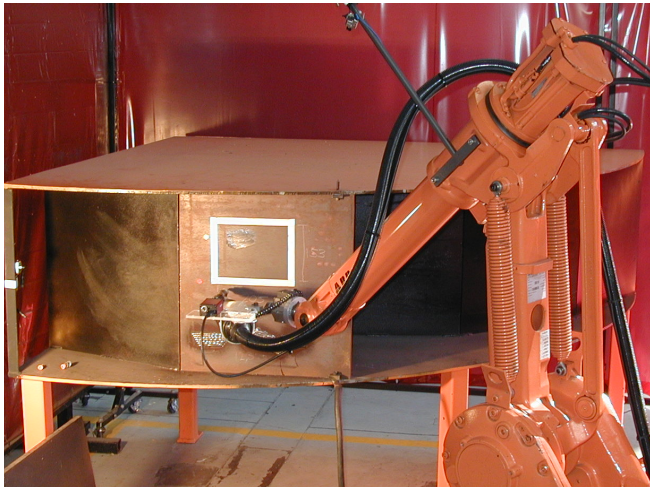
every blade is damaged differently
the surface is a compound 3D curve
manual weld repair is slow and inconsistent

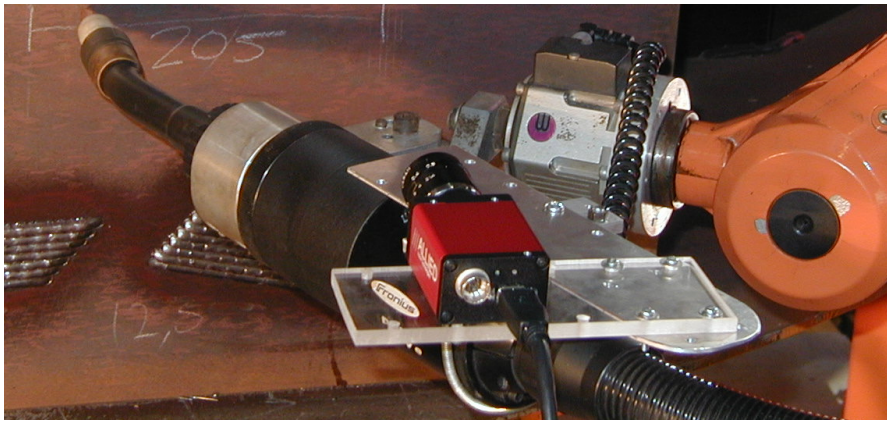
consistent weld quality
higher productivity
the operator leaves the confined space
but: who writes the program for each blade?

Vision-based rapid robot programming

EXAMPLE

the camera writes the program, not the operator





the repair cell, and the camera mounted beside the torch

one hour
to teach a blade by hand

about 10 seconds
from one camera image

At that speed a one-off repair becomes economic — which is the whole point.

Image processing, programming and deposition

EXAMPLE

the three stages of the automated repair



image processing

detect the damaged region and its key points

automatic programming

generate a clash-free path over the 3D surface

metal deposition

weld the layers and inspect the result

The camera does the first stage, and everything downstream depends on how well its geometry is known.

Table of Contents

1. Vision systems and image data
2. Geometric transformations
3. Camera models
4. Calibration for robotic vision
5. Image processing (regions and kernels)
6. Compression (run-length + optional transforms)
7. Recent progress and future of robotic vision (optional)

PART 1

Vision systems and image data

Pixels, colour and resolution

- Image types and digital representation
- Resolution and one-dimensional images
- Camera sensors

Types of images

CORE

a number in every pixel: intensity, or intensity and colour



colour

intensity and colour in every pixel



grey scale

one intensity number per pixel



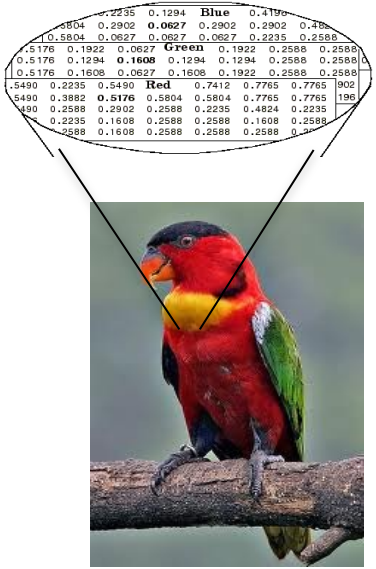
binary

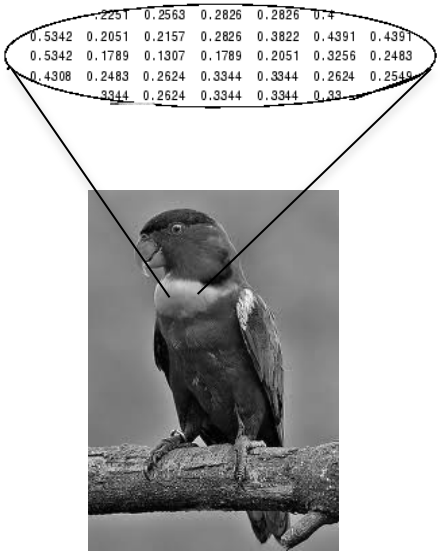
one bit per pixel: 0 black, 1 white

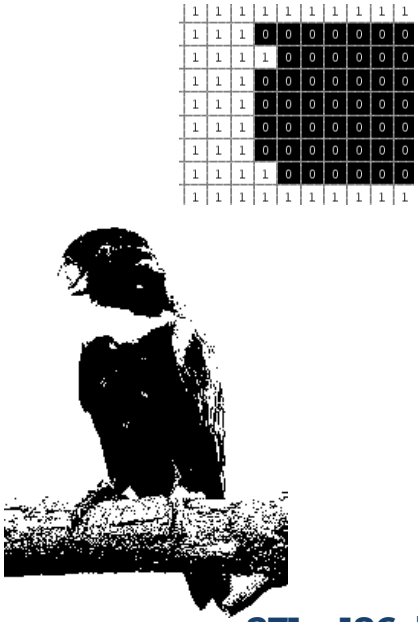
Same scene, three storage formats. Everything later in this lecture assumes the grey-scale case; colour is three of them.

How an image is represented in a computer

CORE







271 × 186 × 3

three 8-bit planes: R, G, B

271 × 186

one 8-bit plane, 0 to 255

271 × 186 logical

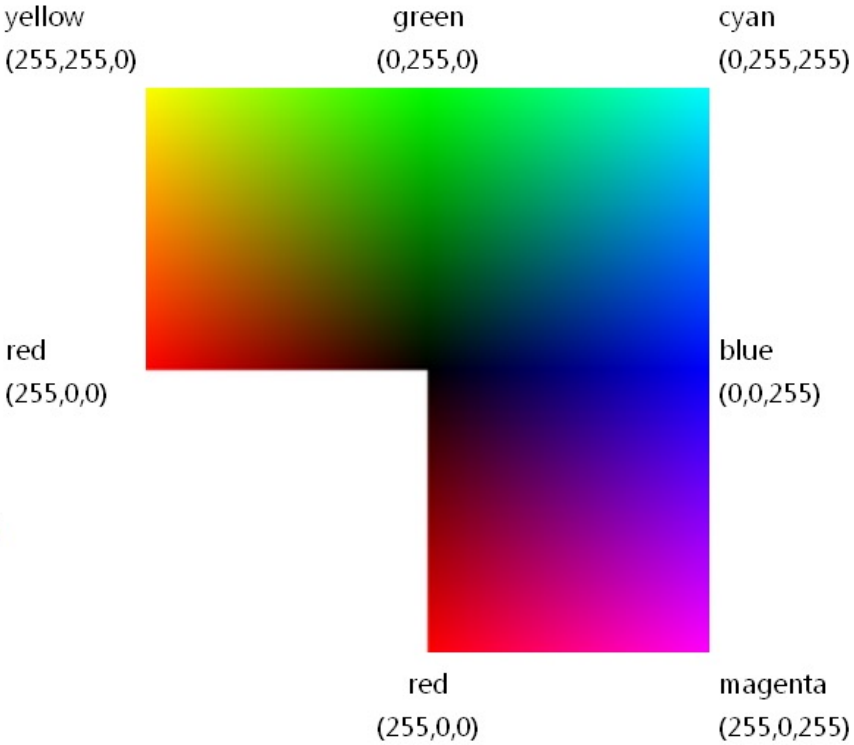
one bit per pixel

The RGB colour model

an additive model: red, green and blue light added together

CORE

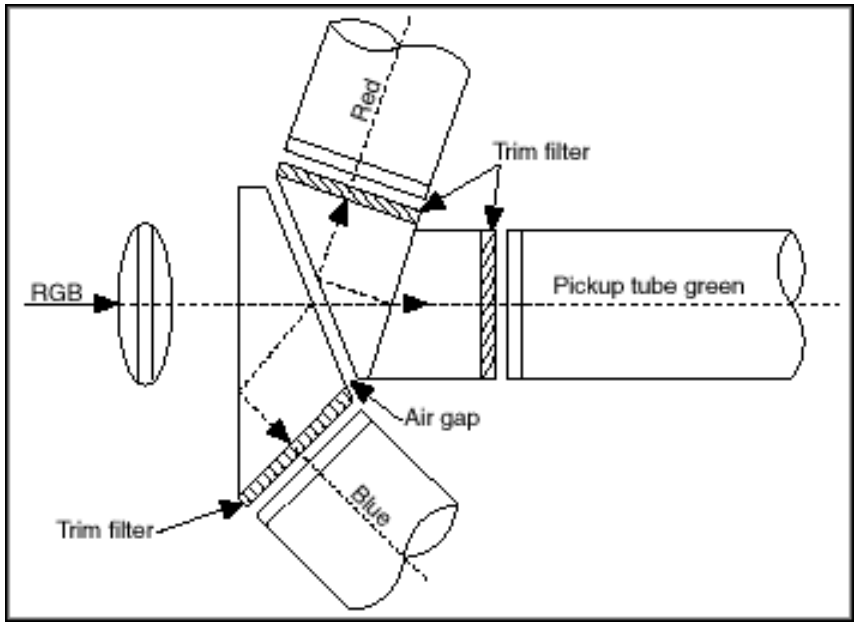
- (0, 0, 0) is black
- (255, 255, 255) is white
- (255, 0, 0) is red
- (0, 255, 0) is green
- (0, 0, 255) is blue
- (255, 255, 0) is yellow
- (0, 255, 255) is cyan
- (255, 0, 255) is magenta



Three-sensor colour cameras

CORE

the highest quality option: split the light, filter each path



a beam splitter sends the light down three paths

each path has its own trim filter and its own sensor

every sensor sees the whole image, in one primary

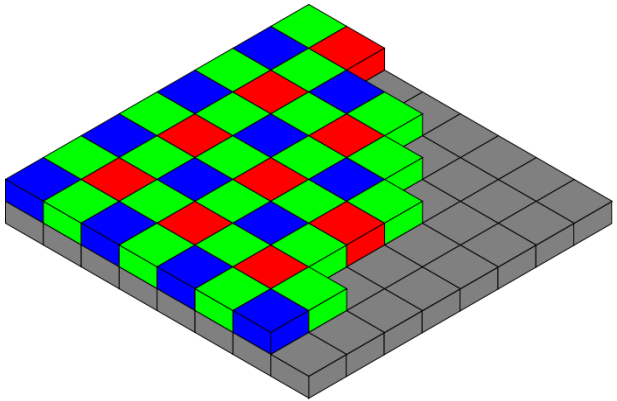
no interpolation, so no colour artefacts

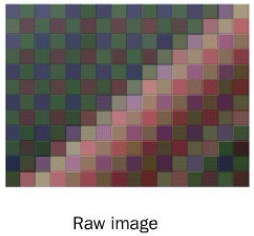
Bulky, expensive, and needs three sensors held in register. Used where colour fidelity matters more than cost.

The Bayer filter and single-sensor cameras

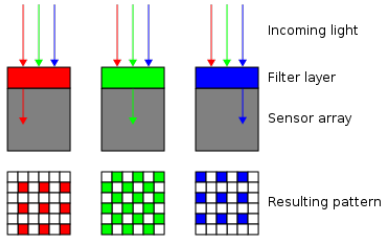
CORE

one sensor, a mosaic of filters, and interpolation





raw mosaic



the three colour planes

a colour filter array sits directly on the sensor

each square of four pixels has one red, one blue and two green

green is doubled because the eye is most sensitive to it

the missing two channels at each pixel are interpolated from the neighbours

One sensor, all three colours captured at the same instant, much cheaper — at the cost of interpolated colour.

One-dimensional images

CORE

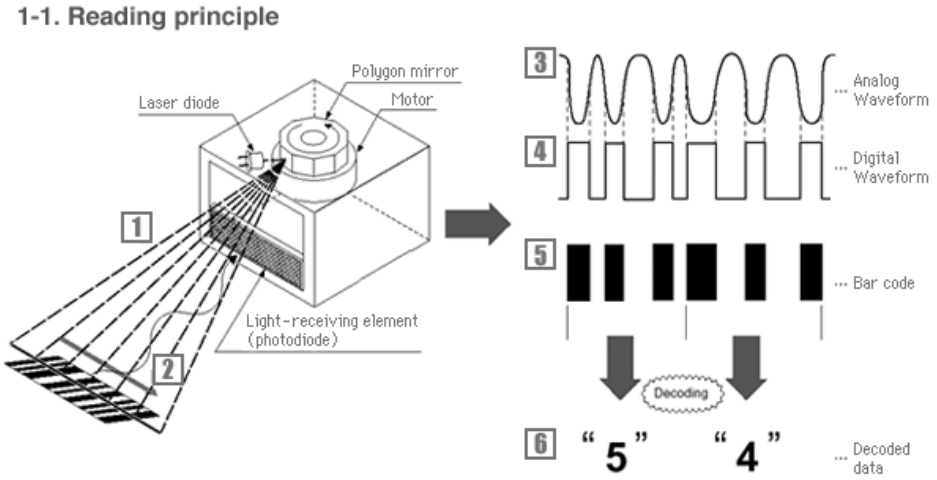
A single intensity profile can support a useful inspection task

A one-dimensional image

A line sensor records intensity only across one direction. Motion, or a scanned laser beam, supplies the other direction when needed.

Typical application

Barcode reading: identify bar widths from one reflected-light profile.



From a single line to an inspection image

The camera repeatedly records one line while material or parts move through the field of view.

Line-scan inspection on a conveyor

CORE

The motion of the material builds the second image dimension

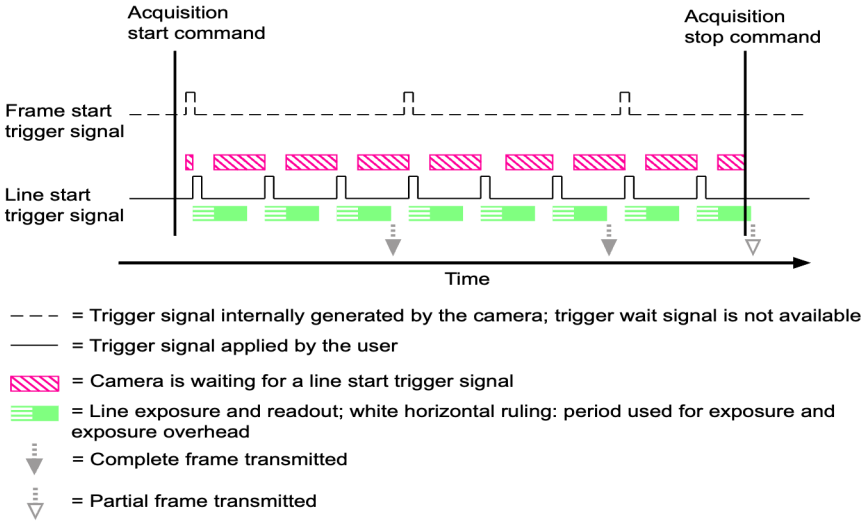
Basler line-scan use cases: https://docs.baslerweb.com/line-scan-gige-use-cases

Why use line scan?

- High resolution across a wide web
- Continuous material such as paper, steel or fabric
- Repeated objects moving along a conveyor

Practical case

Inspect one image per moving object, or join successive frames for a larger object.



Two signals coordinate the capture

A presence trigger identifies the object. A shaft encoder can trigger each line at a fixed travel distance when belt speed varies.

Sampling and conveyor speed

CORE

One short example: choose the line rate to preserve geometry

Along-travel sample spacing = conveyor speed ÷ line acquisition rate

This spacing is separate from across-line pixel spacing, which comes from optics and sensor pixels.

Initial setting

Conveyor speed: 600 mm/s
Line rate: 2,000 lines/s

600 ÷ 2,000 = 0.30 mm/line

Faster conveyor

Conveyor speed: 900 mm/s
Keep 0.30 mm/line

Required line rate = 900 ÷ 0.30 = 3,000 lines/s

At 2,000 lines/s, spacing is 0.45 mm/line: a 90 mm part gives 200 rows instead of 300. At the old display scale it is compressed along travel.

Image resolution

CORE

cell size — the physical size of one pixel

Cell size, the spatial resolution of an image, is the size of each pixel. It can be as small as a few micrometres for close scenes.

device	pixels	total
HD television	1920 × 1080	2.07 million
27-inch computer display	2560 × 1440	3.69 million
Nikon D800 colour image	7360 × 4912	36 million
human eye (estimate)	24k × 24k	~576 million

More pixels is better resolution only if the optics can resolve that detail and the pixel is large enough to collect enough light. Pixel count is a ceiling, not a guarantee.

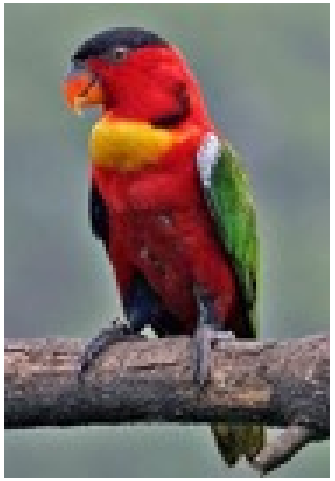
Resolution comparison

CORE

the same photograph at three sampling densities



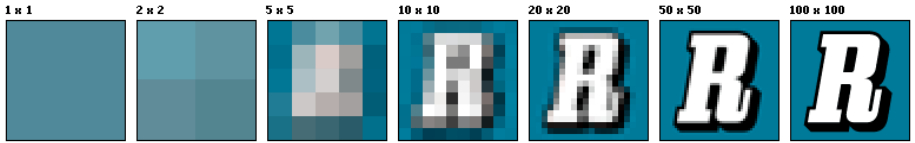
270*186



135*93



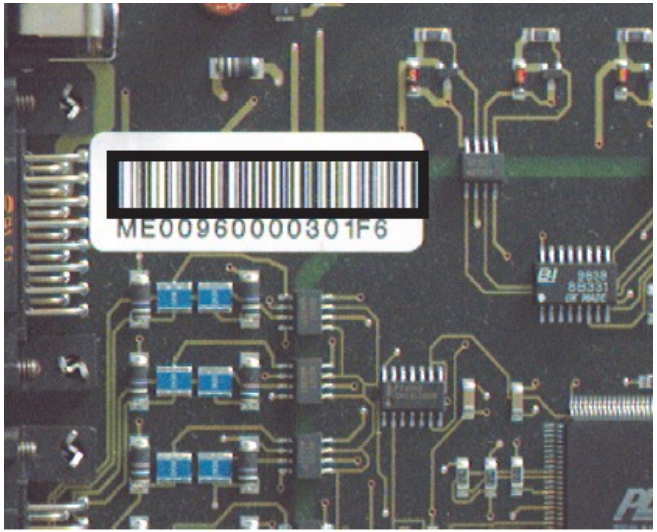
68*46



Physical resolution: millimetres per pixel

EXAMPLE

barcode 30 mm long, camera 640 × 480, sensor 16 × 12 mm



given	value
barcode length	30 mm
camera	640 × 480 pixels
sensor size	16 × 12 mm

pixel pitch on the sensor

$$\frac{16\ mm}{640\ px} = 25\ \mu m$$

A bar narrower than about three pixels cannot be read reliably, so 0.047 mm per pixel sets the narrowest bar this setup can decode — roughly 0.15 mm.

what one pixel covers on the barcode

$$\frac{30\ mm}{640\ px} = 0.047\ mm\ /\ px$$

The machine-vision camera: lens and sensor

CORE

The lens forms the image; the sensor samples it into pixel values



1 Lens
Projects the scene onto the sensor. Its choice sets field of view, working distance, focus and perspective.

2 Image sensor
A pixel array converts light into numbers. Resolution and pixel pitch determine how finely the image is sampled.

The lens–sensor pair determines what the vision system can see. Modern machine-vision cameras normally use CMOS sensors; CCD is older technology that students only need to recognise.

PART 2

Geometric transformations

Coordinate mappings before camera projection

- Euclidean, affine and projective mappings
- Homogeneous coordinates
- Perspective projection and homography

Euclidean transformations

REVIEW

translation and rotation — length and angle are preserved

Euclidean transformations preserve length and angle measure.
Moreover, the shape of a geometric object will not change. That is, lines transform to lines, planes transform to planes, circles transform to circles, and ellipsoids transform to ellipsoids.
Only the position and orientation of the object will change.

$$Tran(p) = \begin{bmatrix} 1 & 0 & 0 & p_1 \\ 0 & 1 & 0 & p_2 \\ 0 & 0 & 1 & p_3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

translation

$$Rot(\varphi, k) = \begin{bmatrix} R_k(\varphi) & 0 \\ 0 & 1 \end{bmatrix}$$

rotation about an axis k

$$T_{Eucl} = \begin{bmatrix} n_x & t_x & b_x & P_x \\ n_y & t_y & b_y & P_y \\ n_z & t_z & b_z & P_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

the general rigid transform: R in the top left, P in the last column

Lines stay lines, planes stay planes, circles stay circles. Only the position and orientation of the object change.

Homogeneous coordinates

CORE

four numbers for a 3D point, so that translation becomes a matrix multiply

a system of coordinates used in projective geometry, as Cartesian coordinates are used in Euclidean geometry

represent the 3D point (x, y, z) as the 4-vector (x, y, z, 1)

$$(x, y, z, 1) \equiv (wx, wy, wz, w), \quad w \neq 0$$

the representation is not unique: any scale factor w ≠ 0 gives the same physical point

in robotics w = 1 is normally used

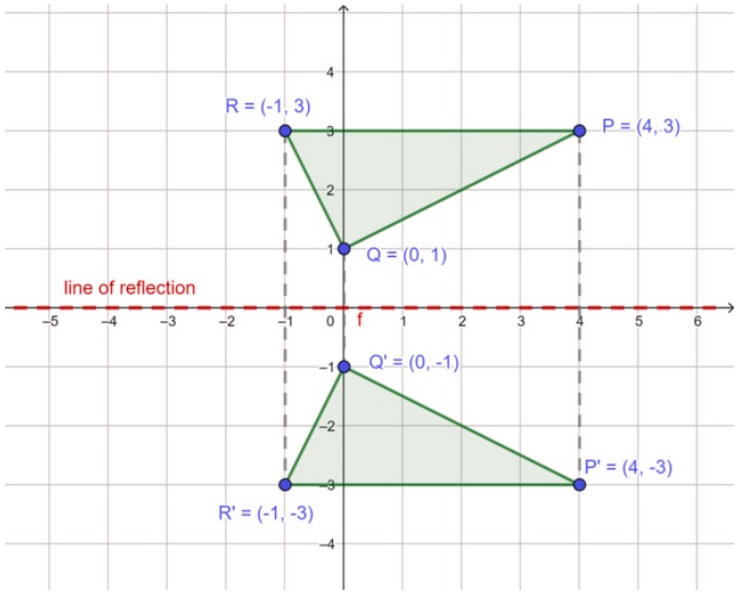
the 3D coordinates come back by dividing through by the fourth component

$$(wx, wy, wz, w) \rightarrow \left(\frac{wx}{w}, \frac{wy}{w}, \frac{wz}{w}\right)$$

Reflection

CORE

An orientation-reversing map, not a rigid-body pose



For the 2D diagram, reflection is easiest to see as a flip across the x-axis. Lengths and angles survive, but the orientation is reversed.

$$T_{Ref,x-axis} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$y \rightarrow -y$$

A mirror or single-axis flip reverses handedness. Ordinary lens inversion flips both image axes: a 180° rotation, not a reflection.

Affine transformations

CORE

a generalisation of Euclidean: lines stay lines, parallels stay parallel



preserved: straight lines, parallelism, ratios of lengths along a line, the midpoint of a segment

not preserved: length, angle, and therefore shape — a circle becomes an ellipse

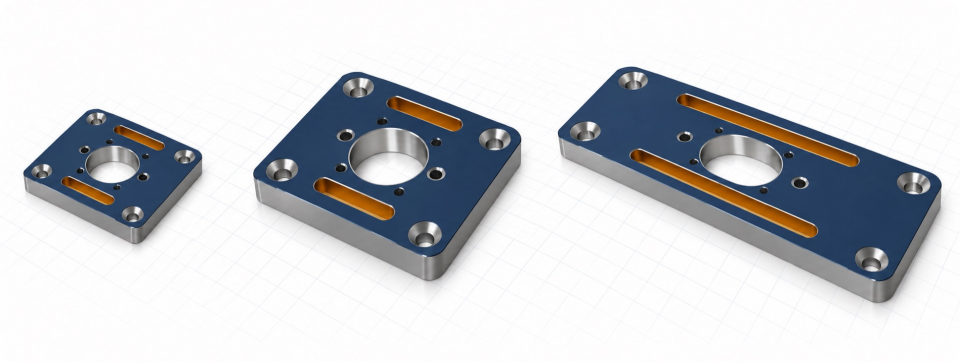
A 2D affine transform can combine scale, shear, rotation and translation. Straight lines remain straight and parallel lines remain parallel.

The next two slides take the two new members one at a time: scaling and shear.

Scaling

CORE

stretch or shrink each axis independently



$$T_{Scale} = \begin{bmatrix} p & 0 & 0 & 0 \\ 0 & q & 0 & 0 \\ 0 & 0 & r & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

one factor per axis

scaling stretches or shrinks the object, so it changes lengths and angles

uniform scaling changes size but not shape — that case is called a similarity transform

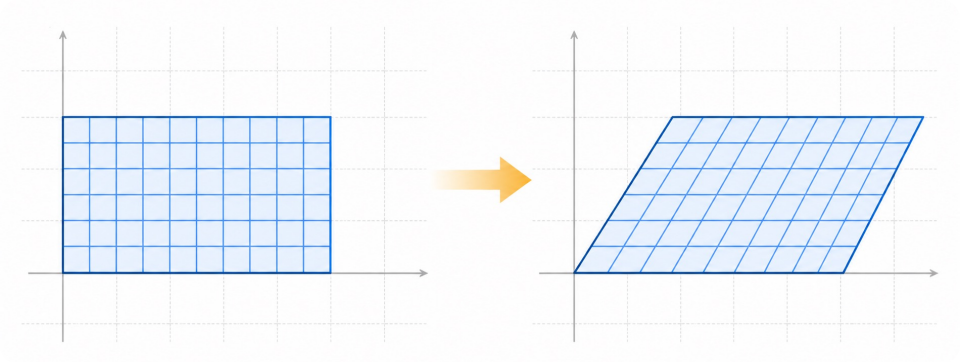
non-uniform scaling turns a square into a rectangle and a circle into an ellipse

A pixel that is not square is exactly a non-uniform scaling between millimetres and pixels — that is where $f_x \neq fy$ in the intrinsic-matrix model comes from.

Shear

CORE

"pushing" the object parallel to a coordinate axis



$$T_{Sxy} = \begin{bmatrix} 1 & 0 & a & 0 \\ 0 & 1 & b & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad T_{Sy} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ c & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

shear in x and y, keeping z

shear in y only

the amount each direction is pushed is set by a shearing factor

a rectangle becomes a parallelogram: the opposite sides stay parallel, the angles do not stay right angles

area is unchanged, because the determinant is 1

In a camera, shear appears if the sensor axes are not exactly perpendicular — the skew term s in the intrinsic-matrix model.

Perspective in images

CORE

things nearby are larger, and parallel lines meet



things nearby appear larger

things far away appear smaller

parallel lines intersect at a point

An affine transformation cannot do this: it keeps parallel lines parallel, by definition. Perspective needs a more general map.

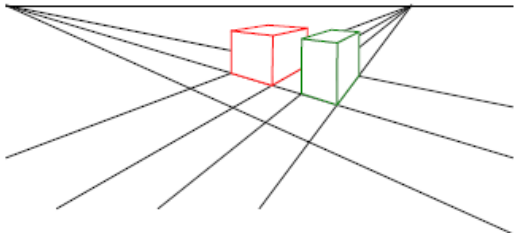
Renaissance painters worked the geometry out by construction; Canaletto's architectural views are the textbook demonstration.

Projective transformation

CORE

the most general linear map in homogeneous coordinates

$$T_p = \begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix}$$



the last row is now free, and that is the whole difference

given a point in homogeneous coordinates (x, y, z, w) and its image (x', y', z', w'), a projective transform has this form

like affine transformations, it does not change the degree of a polynomial

unlike affine transformations, two parallel lines or planes can be mapped to two intersecting ones

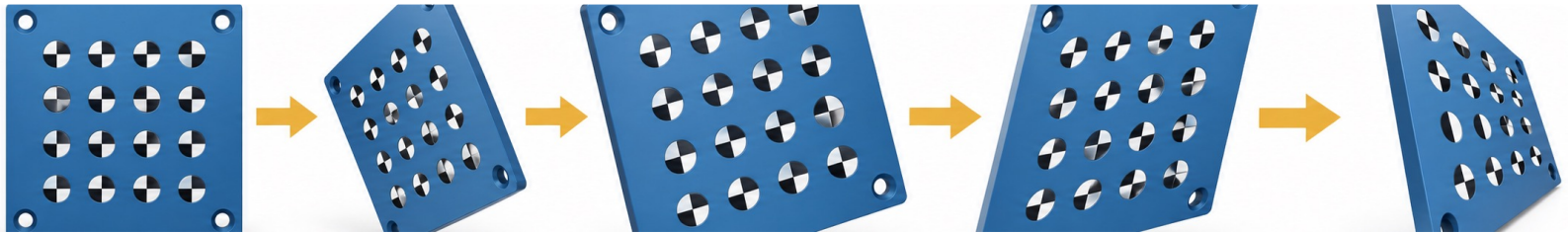
That last property is exactly what a camera does — which is why the camera matrix is projective, not affine.

Hierarchy of 3D transformations

CORE

each row adds freedom and gives up an invariant

name	matrix	DOF	preserves
translation	[I | t]	3	orientation, and all below
rigid (Euclidean)	[R | t]	6	lengths, and all below
similarity	[sR | t]	7	angles, and all below
affine	[A]	12	parallelism, and all below
projective	[H]	15	straight lines

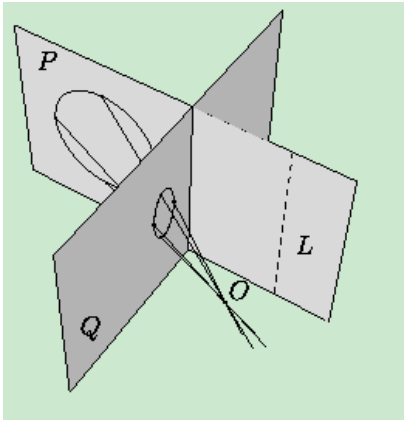


A camera needs the bottom row: it maps a 3D world onto a 2D plane and keeps only straight lines.

Planar perspective transformation

CORE

from one plane to another, with a division at the end



A projective mapping between two planes is a homography H.

Here [x, y, 1] describes a point in planar coordinates.

General 3D camera projection uses P = K[R|t], not this 3 × 3 H.

$$\begin{bmatrix} u \\ v \\ w \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$x' = \frac{u}{w}, \qquad y' = \frac{v}{w}$$

the last step — dividing by w — is the perspective; without it this is just an affine map

Eight free numbers: four point pairs fix a homography.

The operating plane of a homography

CORE

Where one mapping applies

A plane-to-image homography describes features on its calibrated plane.

A fixed mm/pixel scale is a still more restricted approximation.

Perspective changes scale with depth and may change it across the plane.

When the feature is higher

The feature may no longer lie on the assumed plane.

Use its known height with a camera model, a mapping for the correct plane, or measured depth.

Validate at the height used by the robot task.

Before converting pixels to distance, identify the geometric surface on which the measurement lies.

PART 3

Camera models

From scene geometry to image pixels

- Vision system setup and lens models
- Camera coordinates and pixel coordinates
- Intrinsic and extrinsic parameters

Setting up a machine-vision system

CORE

four numbers to choose, and one relationship that links them

camera
the size of the sensor

lens
the focal length

working distance
camera to object

field of view
how much is in the picture



How do we relate an object to its image? 3D → 2D, millimetres → pixels.

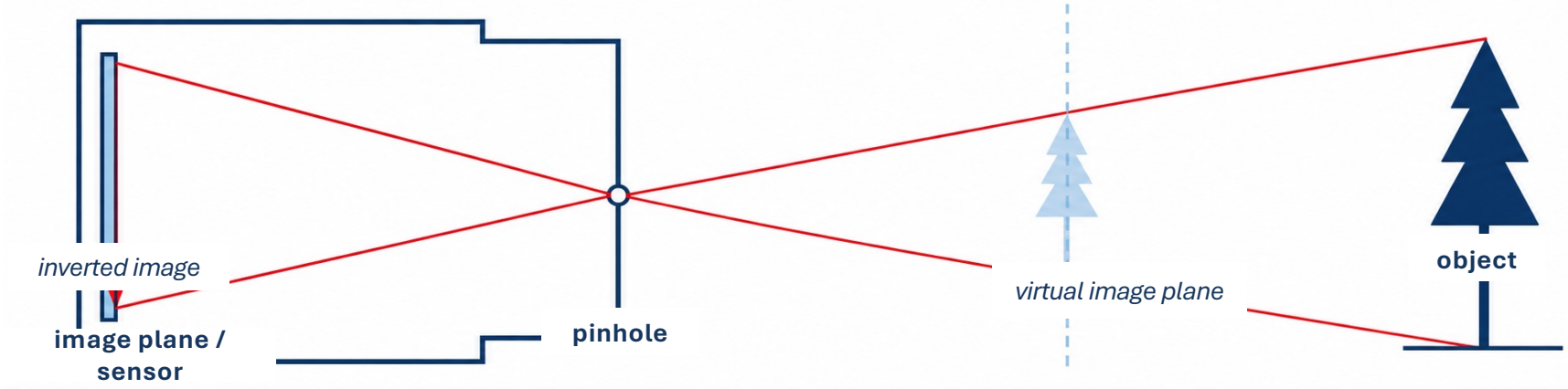
pinhole camera model — the geometry, with no lens

thin lens camera model — adds focus and a finite aperture

The pinhole camera model

CORE

an ideal aperture of zero size, and no lens at all



It describes the relationship between the coordinates of a 3D point and its projection onto the image plane of an ideal camera, where the aperture is a point.

It does not model geometric distortion, the blur of an unfocused object, or the fact that real image coordinates are discrete.

Perspective projection

CORE

the centre of projection at the origin, the image plane at z = f

in camera coordinates the centre of projection is (0, 0, 0) and the projection plane is z = f

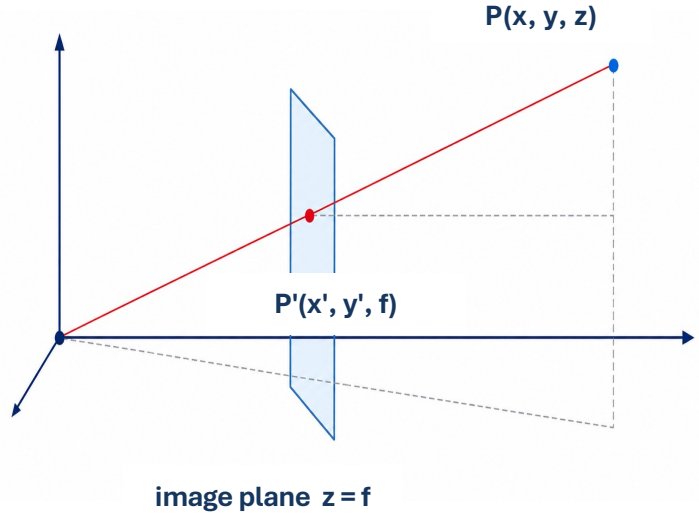
a general point (x, y, z) projects to the plane

$$(x, y, z) \rightarrow (x', y', f)$$

using similar triangles

$$\frac{x}{z} = \frac{x'}{f} \qquad \frac{y}{z} = \frac{y'}{f}$$

$$(x, y, z) \rightarrow \left(f\frac{x}{z}, f\frac{y}{z}, f\right)$$



A height change creates a scale error

EXAMPLE

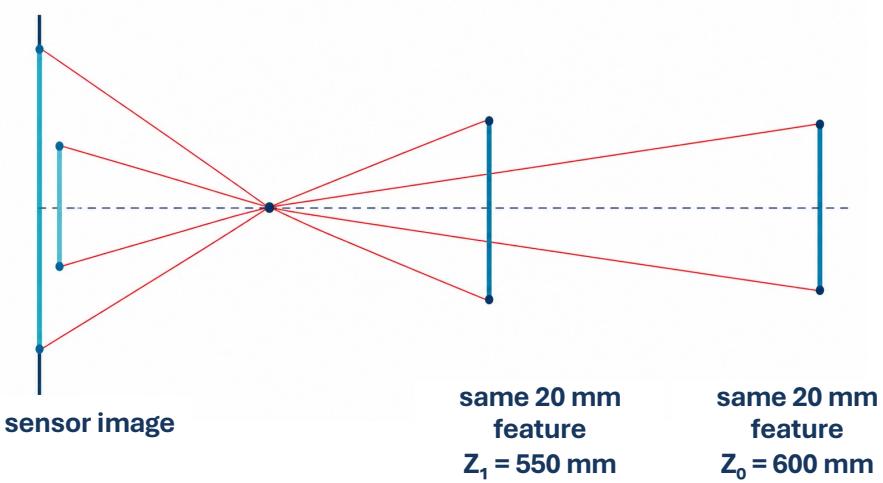
Ideal pinhole example

Calibration plane depth Z_0 = 600 mm.
Feature depth Z_1 = 550 mm.
True feature width = 20 mm.

Pixel width is proportional to width / Z.

Applying the old scale gives:
20 × 600 / 550 = 21.82 mm.

Depth and apparent scale



The nearer feature occupies more sensor pixels.

A well-fitted calibration on one plane does not justify a constant scale at another depth.

Choosing the barcode lens: pinhole model

EXAMPLE

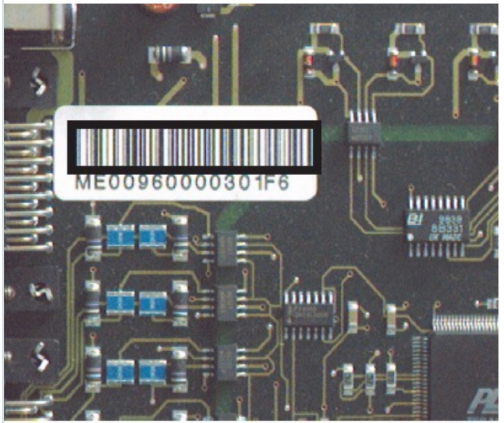
barcode 30 mm at 300 mm, on a 3.2 × 2.4 mm sensor

given	value
barcode length x_o	30 mm
working distance z	300 mm
sensor x_i	3.2 × 2.4 mm
find	a suitable focal length

$$\frac{x_o}{z} = \frac{x_i}{f}$$

similar triangles

$$f = \frac{x_i \cdot z}{x_o}$$



so the barcode just fills the sensor width when

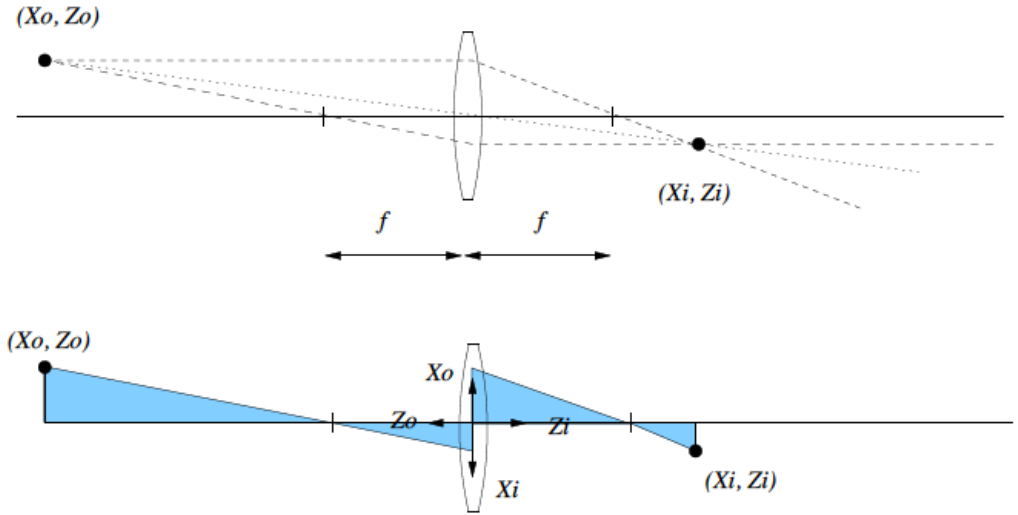
$$f = \frac{3.2 \times 300}{30} = 32\ mm$$

A 32 mm lens is not a catalogue size — take the nearest standard focal length and re-check the field of view.

The thin-lens camera model

CORE

a real lens focuses; the pinhole relationship gains a focal term



$$\frac{1}{z_o} + \frac{1}{z_i} = \frac{1}{f}$$

$$\frac{x_i}{f} = \frac{x_o}{z_o - f}$$

$$\frac{x_o}{f} = \frac{x_i}{z_i - f}$$

Choosing the barcode lens: thin-lens model

EXAMPLE

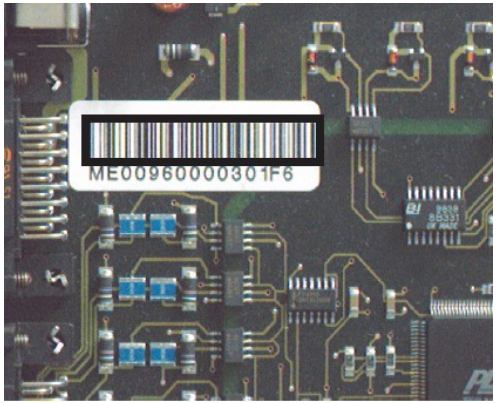
the same setup, solved with the lens equation

given	value
barcode length x_o	30 mm
working distance z_o	300 mm
sensor x_i	3.2 × 2.4 mm

$$\frac{x_i}{f} = \frac{x_o}{z_o - f}$$

rearranged for f

$$f = \frac{x_i \cdot z_o}{x_i + x_o}$$



so with a real lens

$$f = \frac{3.2 \times 300}{3.2 + 30} = 28.9\ mm$$

pinhole 32.0 mm, thin lens 28.9 mm — a 10% difference, because 300 mm is only about ten focal lengths away

Example: Camera Resolution and Distance

EXAMPLE

Canon 500D, 22.3 × 14.9 mm sensor, 4752 × 3168 pixels, 60 mm lens

given	value
sensor	22.3 × 14.9 mm
pixels	4752 × 3168 (15.1 MP)
lens	60 mm focal length
required field of view	27 mm wide

$$\frac{x_i}{f} = \frac{x_o}{z_o - f}$$

the thin-lens relationship again

resolution in the object plane

$$\frac{27\ mm}{4752\ px} = 5.68\ \mu m\ /\ px$$

working distance

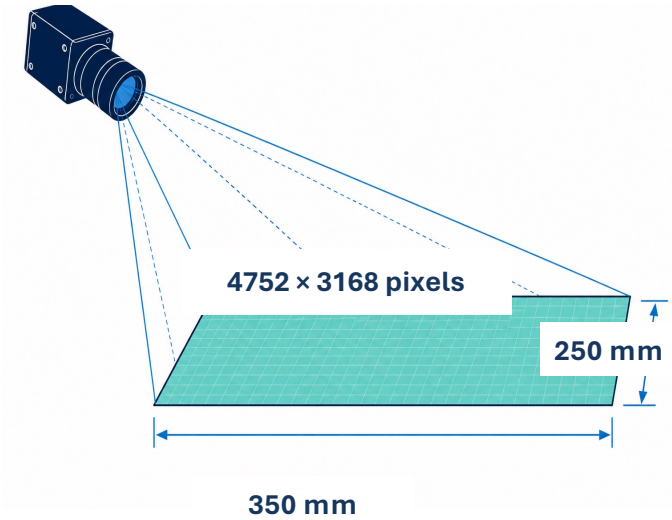
$$z_o = \frac{f\,(x_i + x_o)}{x_i} = \frac{60\,(22.3 + 27)}{22.3} = 132.6\ mm$$

Object-plane sampling is 5.68 µm/pixel. Actual resolution also depends on the lens, aperture, focus and image contrast.

Field-of-view example

EXAMPLE

This example assumes 4752 × 3168 pixels covering 350 × 250 mm. This differs from the 27 mm example.



Direction	Calculation	Nominal object-space sampling
Horizontal	350 / 4752	0.0737 mm/pixel
Vertical	250 / 3168	0.0789 mm/pixel
Coarser directional value	max(0.0737, 0.0789)	About 0.079 mm/pixel

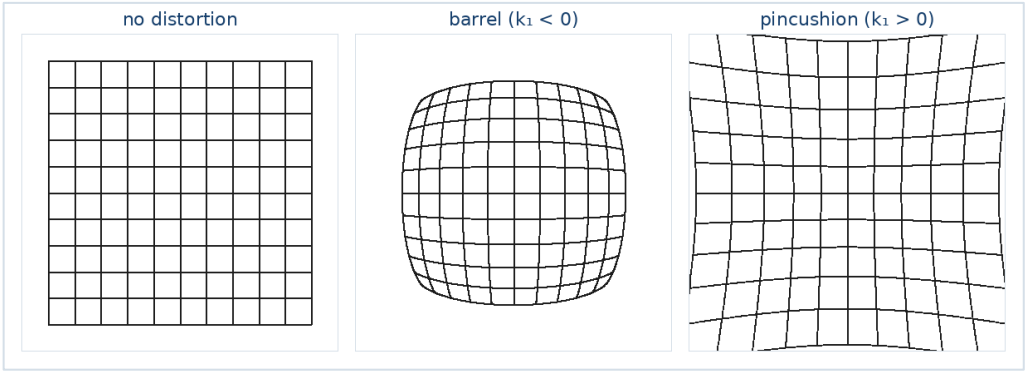
Use the dimensions supplied for each setup. Object-space sampling alone does not establish task accuracy.

Radial lens distortion

CORE

the one part of a real lens the pinhole model cannot ignore





$$r^2 = {x_n}^2 + {y_n}^2$$

$$\begin{bmatrix} x_d \\ y_d \end{bmatrix} = (1 + k_1 r^2 + k_2 r^4 + k_3 r^6) \begin{bmatrix} x_n \\ y_n \end{bmatrix}$$

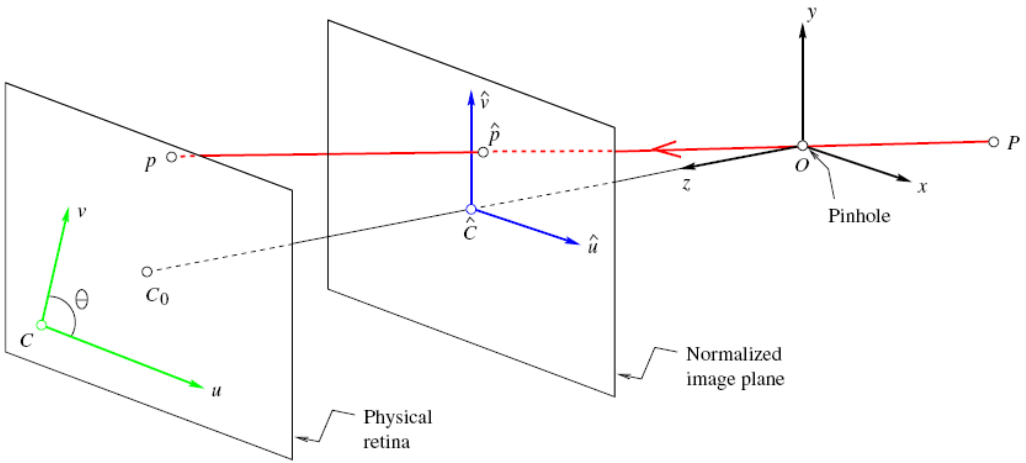
r is the distance from the principal point, in normalised coordinates

Nonlinear, and strongest at the edges of the frame: barrel when $k_1 < 0$, pincushion when $k_1 > 0$.

Camera coordinates and image planes

CORE

World coordinates, normalised rays and physical pixels are different quantities.



1 Camera frame

Express a 3D point relative to the optical centre before applying perspective projection.

$$p_n = \left(\frac{x}{z}, \frac{y}{z}, 1\right)$$

2 Physical image plane

The virtual plane at z = f is measured in metres. A pixel array uses a corner origin.

$$x_i = f\frac{x}{z}, \qquad y_i = f\frac{y}{z}$$

$$u = kf\frac{x}{z}, \qquad v = lf\frac{y}{z}$$

k and l are pixels per metre. Non-square pixels have k ≠ l. The principal-point offset and skew complete the pixel mapping on the next page.

Pixel scale and the intrinsic matrix

CORE

Focal length, principal point and skew collected into a single mapping.

Pixel pitch is 1/k by 1/l metres. Image geometry determines the products α = kf and β = lf, rather than f and pixel pitch separately.

$$u = f_x \frac{x}{z} + s\frac{y}{z} + c_x, \qquad v = f_y \frac{y}{z} + c_y$$

$$K = \begin{bmatrix} \alpha & -\alpha\cot\theta & u_0 \\ 0 & \frac{\beta}{\sin\theta} & v_0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$K = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Derived notation	Modern notation	Meaning
α = kf	f_x	Focal length in horizontal pixels
β / sinθ	f_y	Focal length in vertical pixels
−α cotθ	s	Skew (usually 0 when θ = 90°)
u_0, v_0	c_x, c_y	Principal point measured from the pixel origin

The complete camera model

CORE

A rigid frame change, intrinsic mapping and perspective division.

Extrinsics: world to camera

$$P_c = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} P_w = {}^{c}T_w P_w$$

R and t describe 3 translations and 3 rotations. They change with the camera–world relationship.

Intrinsics: camera ray to pixels

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} R & t \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix}$$

K describes the camera and lens. It remains fixed while focus, zoom and image sampling remain unchanged.

$$q = MP_w, \qquad u = \frac{q_1}{q_3}, \qquad v = \frac{q_2}{q_3}, \qquad \lambda = q_3$$

5 intrinsic + 6 extrinsic parameters = 11 for the general pinhole model. Lens-distortion coefficients are additional. The 3 × 4 camera matrix is M = K[R | t].

PART 4

Calibration for robotic vision

Camera characterization and eye-in-hand calibration

- Camera intrinsics and lens distortion
- Target pose from 3D–2D correspondences
- Eye-in-hand calibration concept

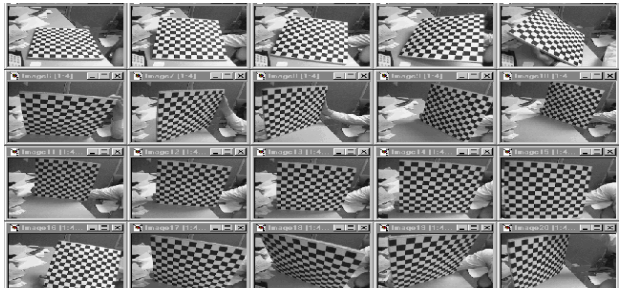
Camera calibration: what is estimated

CORE

The camera model stays fixed while the checkerboard pose changes.

1 Camera and lens calibration

Move a checkerboard through diverse views.
Camera fixed; board moved by hand.
Estimate K and distortion D.
Each image also yields R_i and t_i.



Board–camera pose changes across views.

Python / OpenCV calibration; Corke robot kinematics.

2 Camera–robot relation

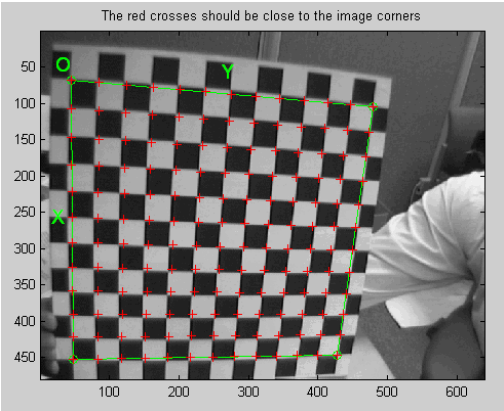
Record the robot pose and board image together.
Solve one fixed camera-to-gripper transform X.
Use X to connect vision and robot coordinates.

Ordinary camera calibration does not require a robot.
Eye-in-hand calibration is introduced after K, D, R and t.

Calibration images and intrinsic estimation

CORE

Known planar geometry supplies constraints on K across differently oriented views.



Checkerboard: sub-pixel corner positions. Circle grid: robust centroids, but perspective can bias their location.

Acquisition and estimation

Use about 15–25 sharp views. Tilt the board in both directions (e.g. 30–45°), cover frame edges and vary distance. Keep focus fixed.

$$H_j = K[r_1 \quad r_2 \quad t]_j$$

For the board plane Z = 0, each view gives a homography and two constraints on K. Three distinct orientations can initialise the fit. Joint nonlinear refinement then reduces reprojection error.

$$f_x = \frac{f}{s_x}, \qquad f_y = \frac{f}{s_y} \quad (pixel\ pitch:\ s_x, s_y)$$

A flat-on board at the image centre is poorly conditioned. Typical output: f_x, f_y, c_x, c_y, with skew normally fixed at zero.

Planar calibration: implement with Python / OpenCV. Original target image retained.

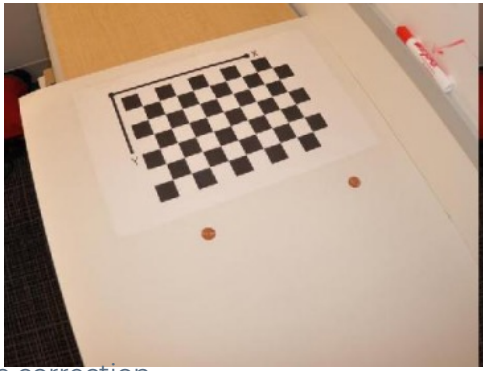
Lens distortion: observation and correction

CORE

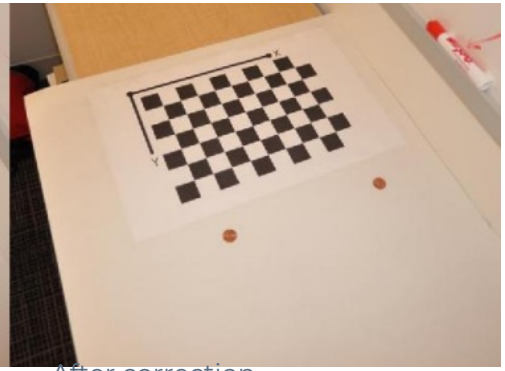
Radial and tangential displacement must match the measured image points.



Straight reference lines expose radial curvature.



Before correction



After correction

$$x_d = x_n L + 2p_1 x_n y_n + p_2(r^2 + 2{x_n}^2)$$
$$y_d = y_n L + p_1(r^2 + 2{y_n}^2) + 2p_2 x_n y_n$$
$$L = 1 + k_1 r^2 + k_2 r^4 + k_3 r^6$$

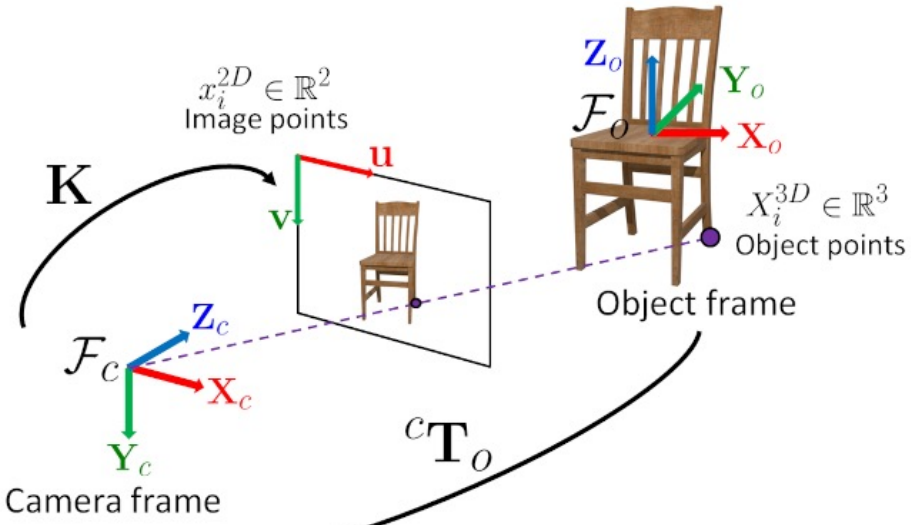
$r^2 = x_n^2 + y_n^2$. Radial terms grow with distance from the principal point. Tangential terms account for decentring / misalignment.

Sources: OpenCV calibration tutorial and MathWorks Camera Calibrator. Estimate coefficients with K; wide-angle lenses may need another model.

PnP: target pose from 3D–2D points

CORE

Known target points, detected image points and K determine R and t for one view.



What is known and what is solved

Known: K, distortion and matching 3D–2D points.
Solved: R and t for this view.
The figure's object o is our target t.

$$\lambda\, m_i = K[R \quad t]M_i$$

$${}^cT_t = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix}$$

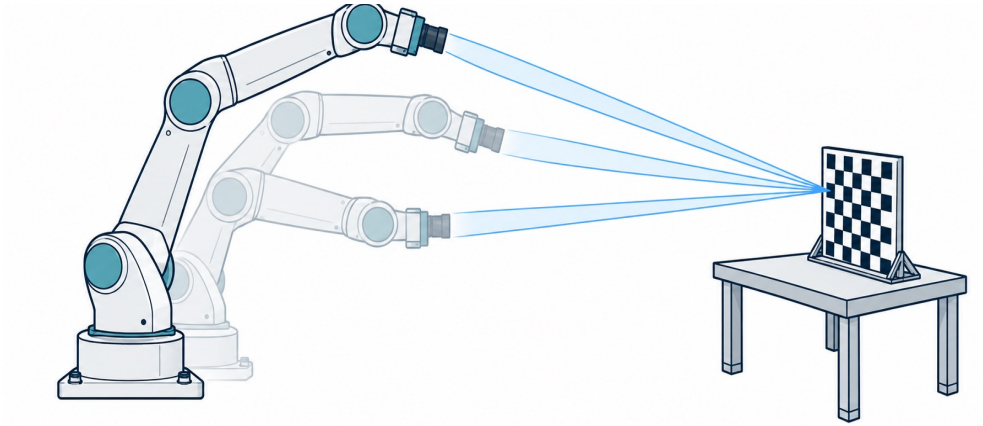
Output: target-to-camera R and t. Check the direction returned by the API.
Practical check: large reprojection error indicates a poor pose estimate.

Source: OpenCV solvePnP. No MathWorks robotics API required.

Eye-in-hand calibration

CORE

The camera moves with the robot while the calibration board remains fixed.



camera fixed to wrist

board fixed

What is recorded

1 Move the robot to several camera poses.

2 At each pose, save the robot pose and observe the fixed board.

3 Solve one fixed camera-to-gripper transform X.

The board stays fixed. The camera moves with the robot. X connects camera measurements to the robot wrist.

Concept based on the OpenCV calibrateHandEye workflow. Visual generated for this lecture, 2026.

OPTIONAL · PART 5

Image processing

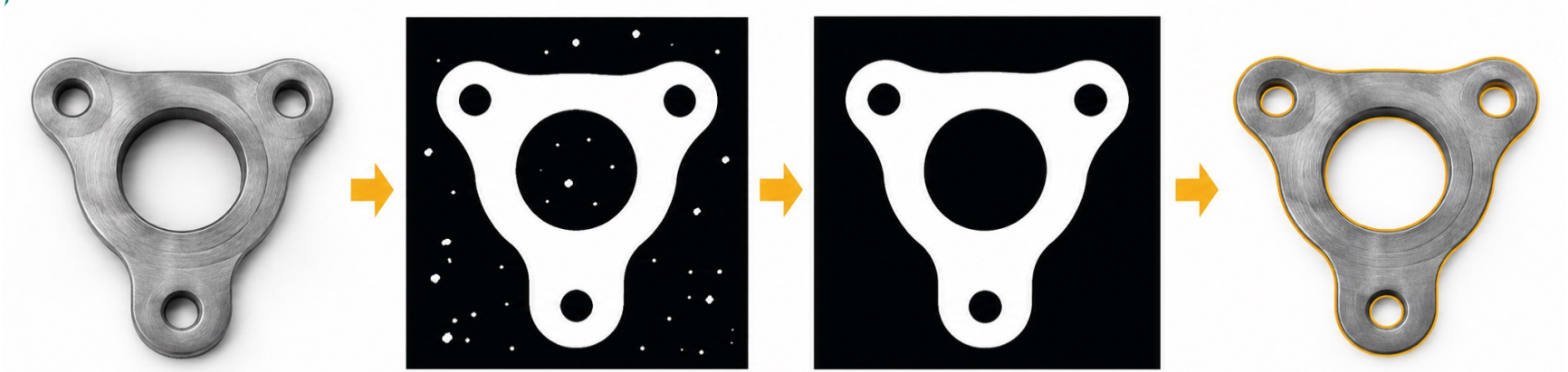
Optional overview: from pixels to simple recognition

- Thresholding and feature measurement
- Template matching and convolution kernels
- Edge detection and spatial filtering

From image to measurable regions

OPTIONAL

Purpose: turn raw pixels into a clean candidate region that can be measured



threshold

separate foreground from background using a global or adaptive rule

morphology

opening removes isolated noise; closing fills small gaps in the region

connected region

retain candidate parts by area, shape or position in the image

The binary mask is the input to measurement, not the final result.

Threshold failure under changing illumination

OPTIONAL

Purpose: show why a threshold that works once may fail when illumination changes.

A fixed threshold example

Foreground: intensity 70.
Background: intensity 150.
Threshold: 100, with dark foreground.

Under changed lighting, the foreground becomes 120 and may disappear from the mask.

Remedies and checks

Stabilise illumination and shield stray light.

Control exposure, avoiding saturation.

Consider adaptive thresholds or more robust features, then test expected conditions.

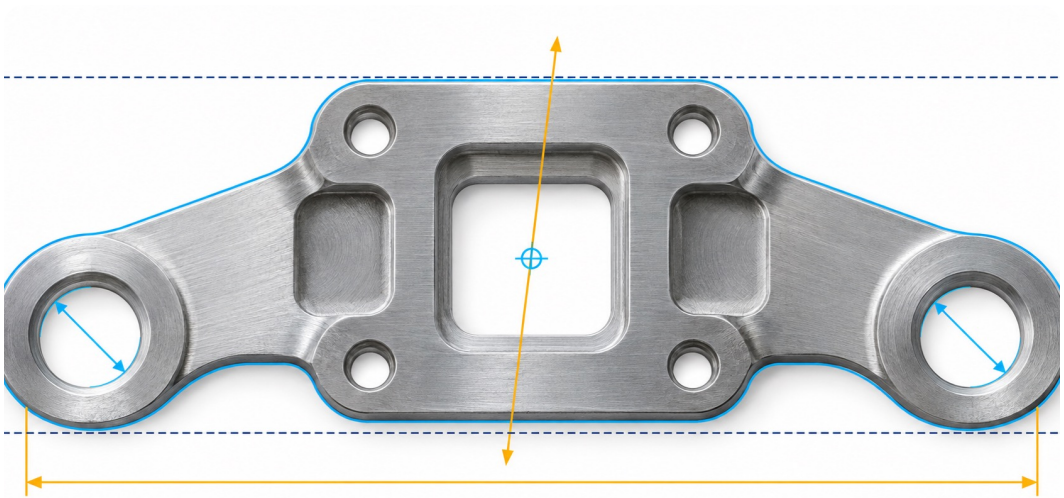
Morphology cannot reconstruct missing measurement evidence reliably.

A segmentation error can shift the measured edge and therefore the robot target.

Feature measurement from a binary region

OPTIONAL

Purpose: convert the segmented region into position, orientation and dimensions.



centroid

$$\underline{u} = \frac{1}{N} \Sigma u_i \quad \underline{v} = \frac{1}{N} \Sigma v_i$$

orientation

principal axis from the second moments of the region

dimensions

area, perimeter, fitted lines and circles, and the bounding box

The measured feature can be checked against a tolerance or passed into the camera-to-robot coordinate chain.

Measure in undistorted coordinates; convert pixels to millimetres only after the camera geometry is known.

Local feature completeness and confidence

OPTIONAL

Purpose: verify that the task-relevant feature was actually observed before accepting the result.

A scan aligns well overall, but part of the edge required for the task is missing.

Check	What it establishes	Possible response
Overall registration residual	Fit of the available correspondences	Inspect distribution and datum agreement
Coverage of the required edge	Whether the local task feature was observed	Acquire another useful view
Feature localisation uncertainty	Confidence at the actual task location	Reject or flag an uncertain target
Independent local verification	Agreement at a task-relevant reference	Release only within acceptance limits

Lecture 08 qualifies the measured geometry. Lecture 09 uses that evidence before generating a path.

Template matching

OPTIONAL

Purpose: locate a known image patch by comparing it with every possible position in the search image.

How the search works

1 Define a template from the object.
2 Move it across the search image.
3 Calculate one score at each position.
4 Select the position with the lowest score.

Two difference scores

$$SAD = \Sigma |I(x,y) - T(x,y)|$$

$$SSD = \Sigma (I(x,y) - T(x,y))^2$$

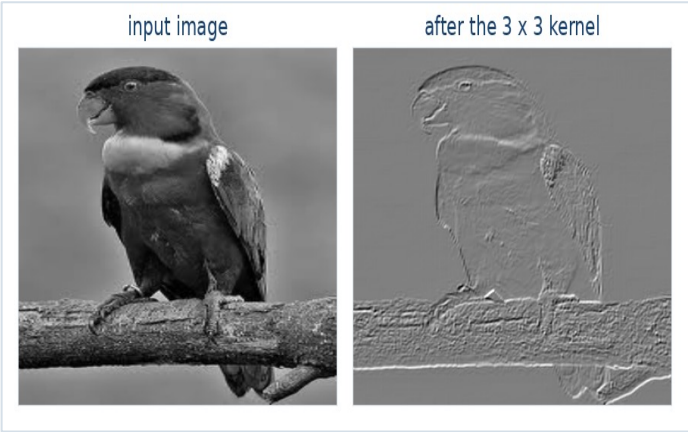
A smaller score indicates a better match.

Limitation: pixel templates work best when scale, rotation and illumination remain close to the stored example.

A 3 × 3 kernel: edge detection

OPTIONAL

the same sliding-window idea, with a small kernel instead of a template



$$w = \begin{bmatrix} -1 & -1 & 0 \\ -1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}, \quad c = 3, \quad b = 128$$

a diagonal difference operator

The offset b = 128 puts zero difference in mid-grey, so both signs of edge are visible.

$$g(x,y) = \frac{1}{c}\left(\sum_{i=-1}^{1}\sum_{j=-1}^{1} w(i,j)\, f(x+i,\ y+j)\right) + b$$

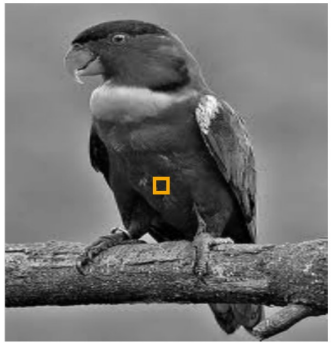
the kernel is multiplied point by point with the neighbourhood, summed, divided by c and offset by b

Template-matching example

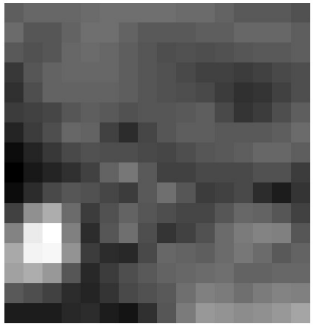
OPTIONAL

finding a known patch, and reading the score map

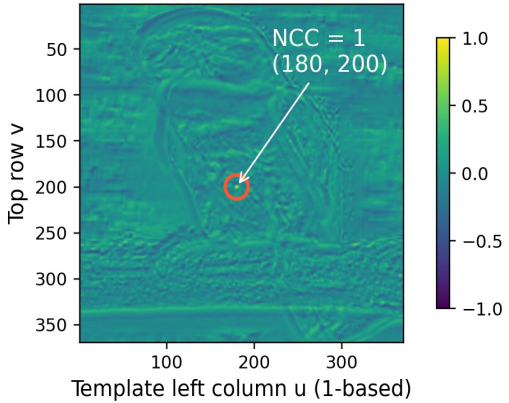
Source ROI: 385 × 384 pixels



16 × 16 patch



NCC score: maximum is the match



NCC example

16 × 16 patch: NCC = 1 at column 180, row 200 (MATLAB indices).

Separate SAD exercise

Image: 420 × 300 pixels
Template: 60 × 60 pixels
87,001 windows × 3,600
Total: 313,203,600

Score convention

NCC: maximum is best.
SAD: minimum is best.
Pixel similarity only.

Limitations of pixel-based template matching

OPTIONAL

a small change in the scene is a large change in the score



the same template, scored against four versions of the same image — lower is better

A small error in orientation or position leads to failure to match. A mask can be rotated or repositioned to compensate, but that multiplies an already expensive search.

So pixel-by-pixel matching is used for objects in known orientations, or for a small region of interest inside a part that has already been located.

Other pattern-recognition methods

OPTIONAL

Purpose: choose a recognition method that matches the expected variation and the required output.



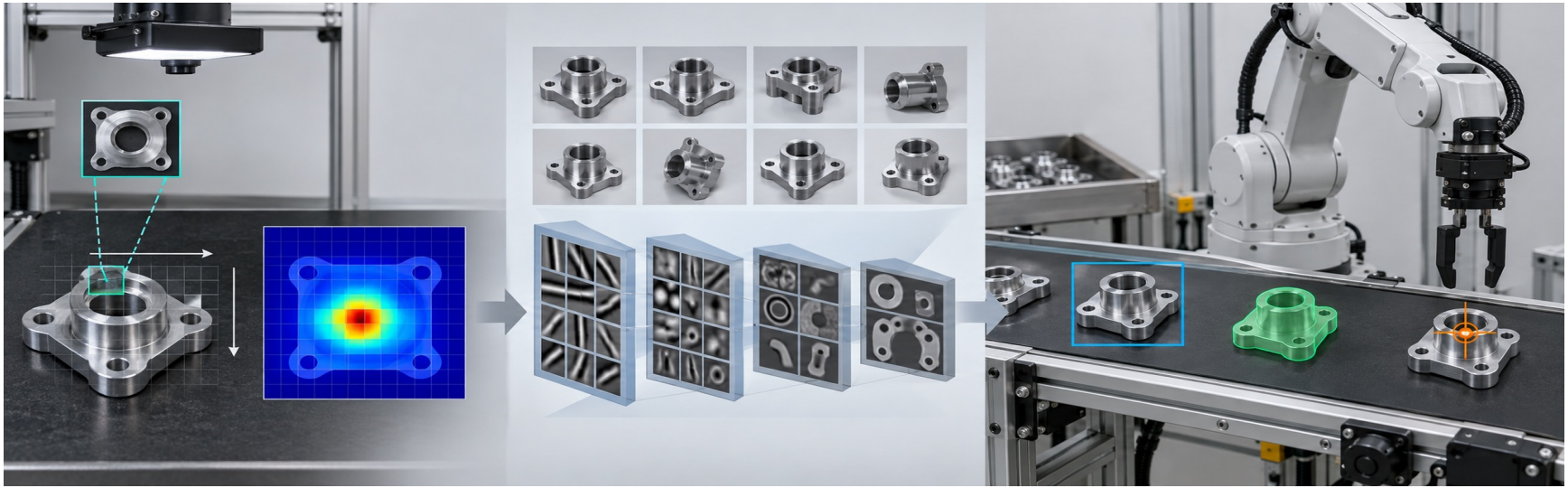
Shape-based edges & contours	Local keypoints scale & rotation	Barcode / OCR structured symbols	Object detection class + location	Segmentation pixel-level region

More invariance and richer outputs usually require more data, computation and validation.

From template matching to CNN-based vision

OPTIONAL

CNNs learn features from data, then support classification, detection and segmentation.



Template matching
Fixed pixel pattern; controlled pose and lighting

CNN feature learning
Edges, textures and object parts learned from examples

Robot-ready outputs
Class, bounding box, mask and grasp location

PART 6

Image compression

Run-length coding, then optional transform compression

- Core: run-length encoding and storage savings
- Optional: Fourier, DFT and DCT
- Optional: JPEG, quantisation and reconstruction

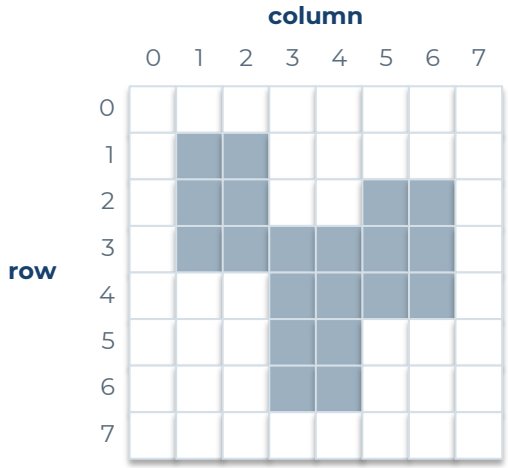
Run-length coding

CORE

store the transitions, not the pixels

a three-element data structure stores the transition information

[the column where the run starts, the length of the run, the pixel value]



a the starting column number

b the length of the run

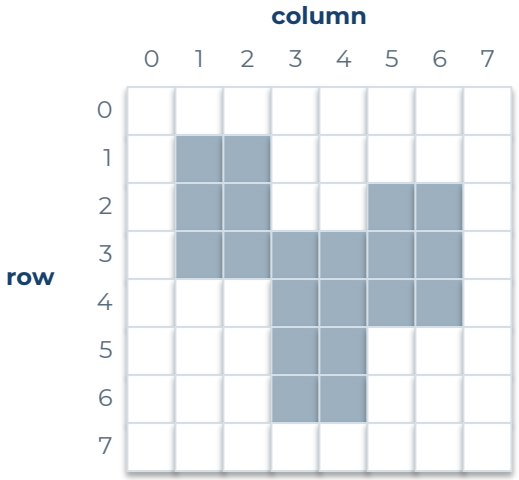
c the pixel value

Only useful when runs are long: documents and graphics, not photographs.

Run-length coding: worked example

EXAMPLE

an 8 × 8 binary image, row by row



the index column is where each row's triplets start in the stored list

row	index	triplets
0	0	[0,8,0]
1	1	[0,1,0] [1,2,1] [3,5,0]
2	4	[0,1,0] [1,2,1] [3,2,0] [5,2,1] [7,1,0]
3	9	[0,1,0] [1,6,1] [7,1,0]
4	12	[0,3,0] [3,4,1] [7,1,0]
5	15	[0,3,0] [3,2,1] [5,3,0]
6	18	[0,3,0] [3,2,1] [5,3,0]
7	21	[0,8,0]

22 triplets in total

What is the storage saving?

CORE

the legacy question, answered

stored as one byte per pixel

$$8{\times}8{\times}1\ B = 64\ B$$

RLE: payload only

$$22{\times}1\ B = 22\ B$$

a triplet fits in one byte: 3 bits for the start column, 4 bits for the run length, 1 bit for the value

Payload

$$\frac{64 - 22}{64} = 0.66$$

22 B payload + 8 B row index = 30 B.
Saving: 53.125% (before headers).
Bit-packed image: 8 B.

mean run length in this image

$$\frac{64\ px}{22\ runs} = 2.9\ px$$

RLE helps long runs; bit-packing is smaller for this image.

Two equivalent run-length representations

CORE

Example row: 0 0 1 1 1 0 0 0. Start at Column 0 and state the convention.

Convention	Representation	Information carried
Value, length	(0,2), (1,3), (0,3)	Compact notation, interpreted in row order
Start, length, value	(0,2,0), (2,3,1), (5,3,0)	PPT notation, with explicit starting columns

Both reconstruct the same row. Storage comparisons must include the bit allocation and any row index.

Image compression

CORE

lossless and lossy, and when each is appropriate

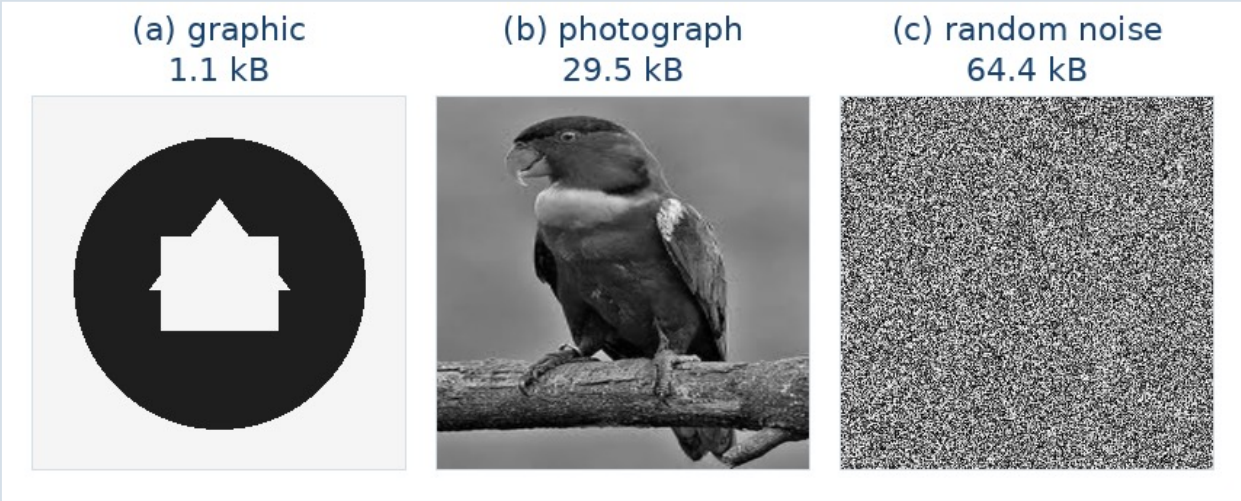
	lossless	lossy
what it does	rebuilds the original exactly	discards data that will not be missed
use when	loss is unacceptable	some loss is acceptable
formats	PNG, GIF, BMP, ZIP	JPEG, HEIC, AVC, HEVC
algorithms	run-length, Lempel–Ziv, Huffman	transform coding, DCT, wavelet
drawback	larger files	quality loss, and it cannot be undone

For machine vision: never measure on a lossy image. JPEG artefacts move edges by a fraction of a pixel, which is exactly the accuracy a calibrated system is trying to deliver.

Why images can be compressed

CORE

because neighbouring pixels are not independent



the same 256 × 256 grid, stored as PNG — uncompressed, all three are 64 kB

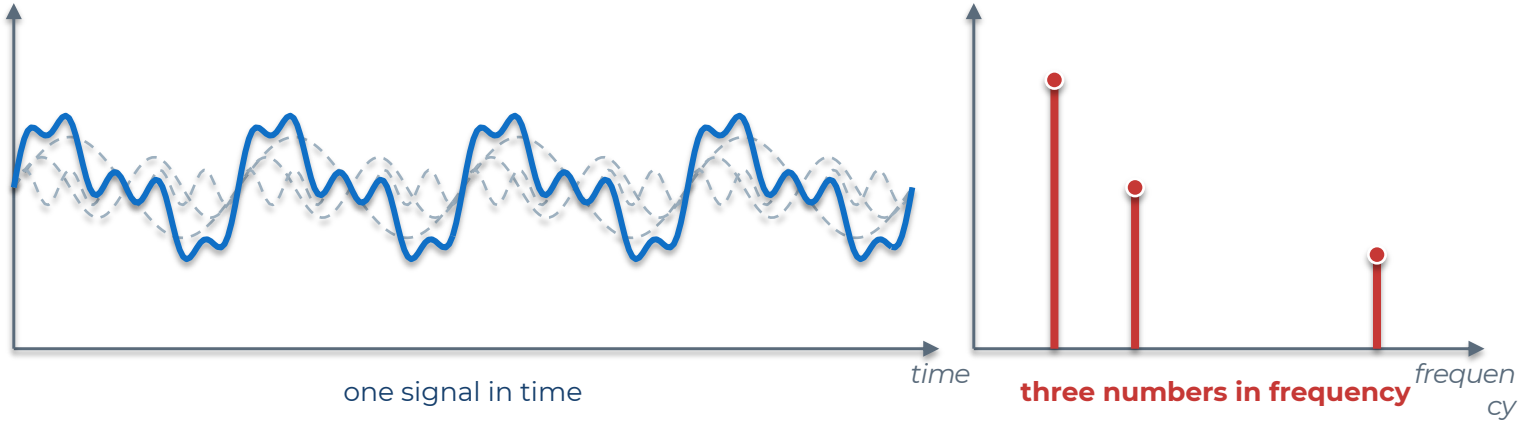
(a) and (b) have structure: a pixel is largely predictable from its neighbours.

(c) has none: every pixel is independent, so PNG makes it slightly larger.

The Fourier transform

OPTIONAL

the same signal, described by frequency instead of by time



one signal in time

three numbers in frequency

$$F(j\omega) = \int_{-\infty}^{\infty} f(t)\, e^{-j\omega t}\, dt$$

$$e^{j\theta} = cos\theta + jsin\theta$$

The DFT and the DCT

OPTIONAL

the discrete versions: complex exponentials, or real cosines

The Discrete Fourier Transform uses a set of harmonically related complex exponential functions, while the Discrete Cosine Transform uses only real-valued cosine functions.

DFT

$$X(k) = \sum_{n=0}^{N-1} x(n)\, e^{-j2\pi kn/N}$$

DCT — the second of the four standard forms, and the one JPEG uses

$$X_k = \sum_{n=0}^{N-1} x_n \cos\left[\frac{\pi}{N}\left(n + \frac{1}{2}\right)k\right], \quad k = 0, \ldots, N-1$$

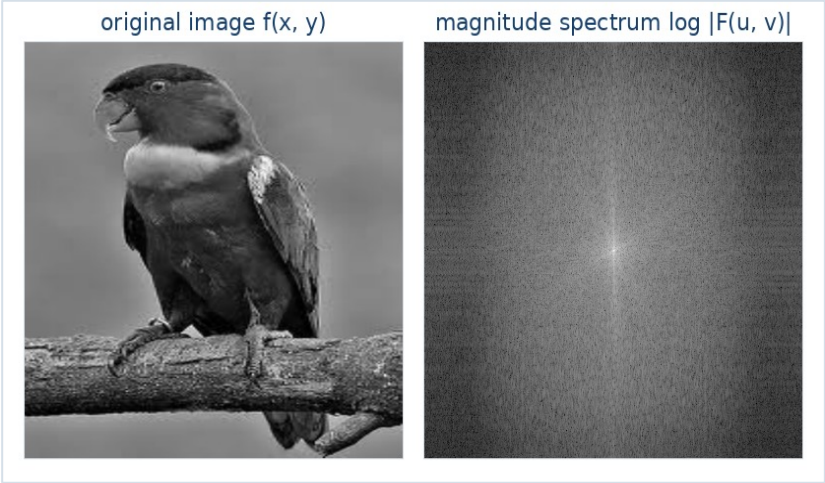
The DFT produces complex numbers: magnitude and phase.

The DCT produces real numbers, and packs the energy of a smooth block into fewer of them — which is why JPEG uses it.

2D FFT: the original image

OPTIONAL

transform every row, then every column of the result



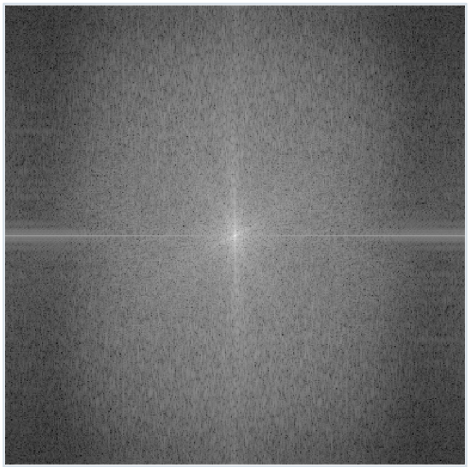
$$F(u,v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x,y)\, e^{-j2\pi\left(\frac{ux}{M}+\frac{vy}{N}\right)}$$

separable: the two-dimensional transform is a one-dimensional transform applied twice

Reading the frequency spectrum

OPTIONAL

where the energy sits, and what each region means



```
% Run in Content/MATLAB Examples
A=double(imread('fft_source.png'));
A=A(39:422,22:406,:); B=A; % bird ROI
if size(A,3)==3
    B=.299*A(:,:,1)+.587*A(:,:,2)+.114*A(:,:,3);
end
F=fftshift(fft2(B));
imagesc(log(1+abs(F))); axis image; colormap gray;
```

the centre low frequencies: overall brightness and broad shading

the edges high frequencies: fine texture and sharp edges

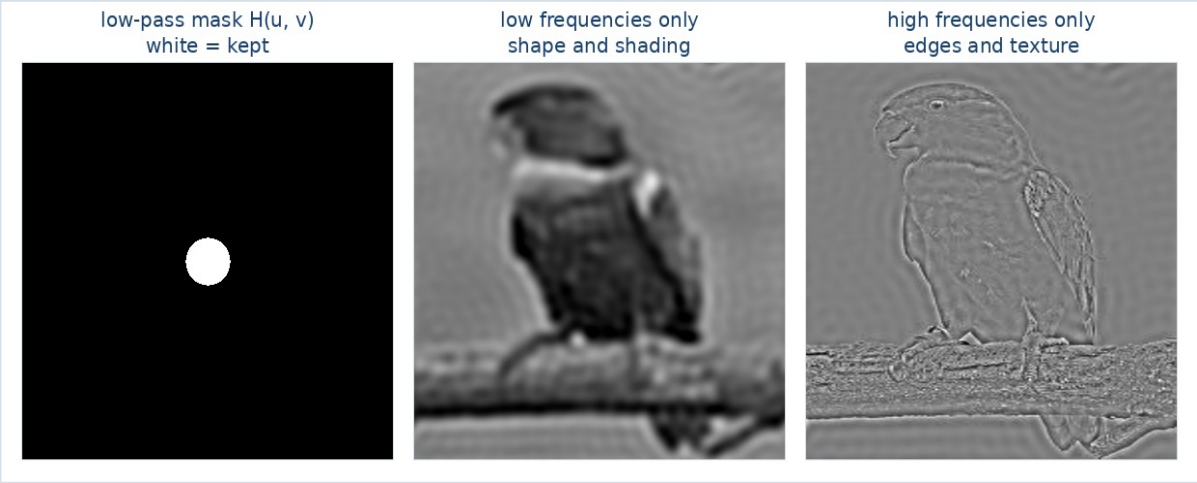
the bright cross the image is not periodic, so the borders act like a step

A smooth image often concentrates energy near the centre. Compression can exploit this, but the exact distribution depends on image content and boundaries.

Frequency-domain filtering

OPTIONAL

multiply by a mask, then transform back



multiply the spectrum by a mask

$$G(u,v) = H(u,v)\,F(u,v)$$

transform back to an image

$$g(x,y) = F^{-1}\{\,G(u,v)\,\}$$

Reconstruction: how few coefficients are enough

OPTIONAL

the same image, rebuilt from progressively fewer coefficients



100%	10%	1%	0.2%
the original	no visible difference	soft, but every feature is still there	ringing and blocking become obvious

The JPEG compression process

OPTIONAL

five stages, two of which throw information away



colour space conversion: R, G, B becomes Y, Cb, Cr, because the eye is far more sensitive to brightness than to colour

chrominance sub-sampling: the Cb and Cr planes are stored at half resolution

the image is divided into 8 × 8 blocks and each is transformed by the DCT

quantisation: each coefficient is divided by a table entry and rounded — this is where the loss happens

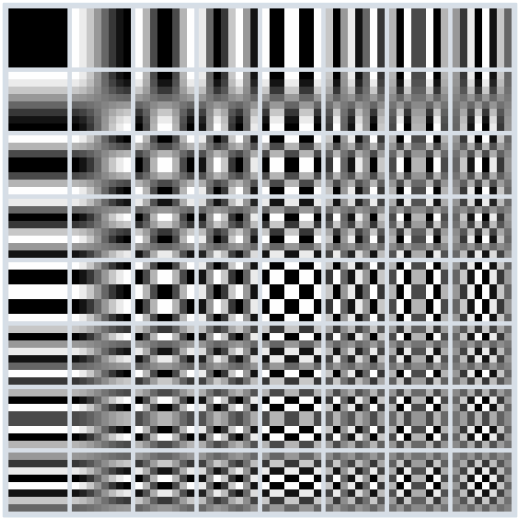
run-length and Huffman coding pack the result losslessly

Only stages 2 and 4 are lossy; the quantisation table is what the "quality" slider changes.

DCT and quantisation

OPTIONAL

any block is a weighted sum of these 64 patterns



the DCT basis for 8 × 8 blocks

$$F(u,v) = \frac{1}{4} C(u) C(v) \sum_{x=0}^{7} \sum_{y=0}^{7} f(x,y) cos \frac{(2x+1)u\pi}{16} cos \frac{(2y+1)v\pi}{16}$$

the two-dimensional DCT of one 8 × 8 block

quantisation: divide by the table, then round

$$\hat{F}(u,v) = round\left(\frac{F(u,v)}{Q(u,v)}\right)$$

low frequencies are top-left, high frequencies bottom-right

the quantisation table divides the high frequencies by large numbers, so they round to zero

long runs of zeros are then cheap to store

JPEG: one 8 × 8 block

OPTIONAL

from 64 numbers to 7

the 8 x 8 block

83	91	86	84	85	84	85	82
83	85	83	79	80	86	82	77
85	83	77	70	67	69	70	69
79	73	71	68	66	65	71	79
71	69	72	75	76	75	75	77
69	73	72	71	73	77	76	72
72	67	64	64	65	69	73	73
70	55	62	70	72	68	64	63

DCT coefficients

-431	3	7	4	0	1	1	0
43	12	3	-4	-9	-5	-3	-1
9	0	-8	6	-1	1	-1	0
20	-11	-6	-2	-6	-3	-3	-1
-1	-4	-5	-2	9	1	0	2
-6	3	13	-4	0	-1	-2	0
-2	7	-4	0	-1	-1	-1	0
-1	2	0	0	0	0	-1	0

after quantisation

-27	0	1	0	0	0	0	0
4	1	0	0	0	0	0	0
1	0	0	0	0	0	0	0
1	-1	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

rebuilt block

88	87	86	85	85	86	88	89
86	85	83	81	79	79	79	80
83	81	78	75	73	71	71	71
77	75	73	71	70	70	70	71
70	70	69	69	70	72	75	76
67	67	67	68	70	73	76	78
67	66	66	66	67	70	72	73
68	67	66	64	64	65	66	67

the block

64 grey values, mostly similar

after the DCT

energy collected in the top-left corner

after quantisation

7 non-zero coefficients out of 64

rebuilt

largest error 12 grey levels out of 255

PART 7

Recent progress and future of robotic vision

Optional extension · not examined

Known rigid objects: vision, geometry and control

Deformable objects: learned perception and action

Open-ended tasks: reasoning, constraints and verification

Three levels of robotic vision

OPTIONAL

Optional extension, not examined. Key question: is the uncertainty in the object, the action or the goal?



New shape, clear task?
A 3D camera and grasp planner may handle an unfamiliar rigid part.

Need to identify it?
A detector or vision–language model (VLM) can label the object.
Depth and calibration locate it.

Need to choose its role?
An agent can propose a support or a cover. Check size, strength and safe reach before moving.

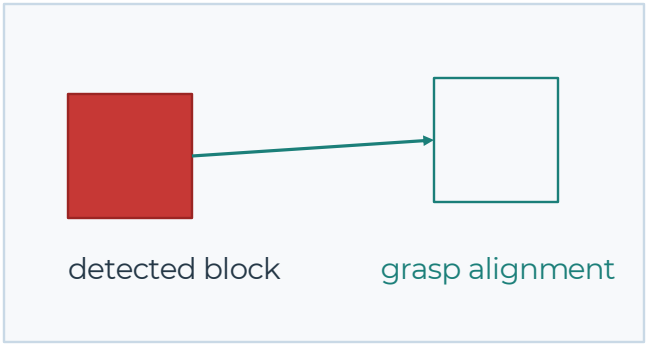
These are typical roles, not exclusive solution sets. A new object does not automatically require VLA.

Known red block: vision and control

OPTIONAL

Task: pick the specified rigid block and place it in a known tray

What the camera measures



Known before the run

Part shape and allowed grasp.
Destination and placement tolerance.
Calibrated camera and robot frames.

1 Find the correct block

Separate red pixels from the background.
Check shape so a red label is not mistaken for a block.

2 Measure where to grasp

Use calibrated geometry or depth to estimate position and orientation in the robot base frame.

3 Approach and correct alignment

Apply the grasp offset and plan a clear approach.
If needed, update from images during the motion.

4 Check that the grasp succeeded

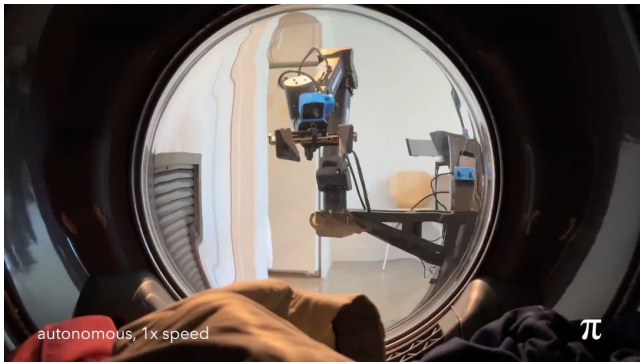
Check gripper width/force and the new image.
An empty grasp triggers re-detection or a safe stop.

A fixed target and measurable error favour conventional vision. Repeated visual correction is visual servoing.

Crumpled towel: a learned action skill

OPTIONAL

Task: take one towel, unfold it and lay it flat



π0 laundry demo (Physical Intelligence)

When a VLA is worth considering

The same goal needs different grasps and pulls as the cloth changes shape.
Relevant skills or demonstrations exist.

What vision needs to reveal

A visible edge, the current fold and whether the cloth moved with the gripper after a small pull.

What the policy must learn

Choose an exposed edge, lift or pull, then inspect.
Regrasp if the corner slips or another layer is caught.

Why one fixed path is insufficient

The useful grasp point changes after each pull.
A successful motion in one fold may fail in another.

What makes deployment credible

Data must cover relevant cloth states and recoveries.
Test new fabrics, views and grippers before deployment.

VLA is one option for learning actions. Model-based cloth control and specialised learned policies are alternatives.

Unclear object roles: task reasoning

OPTIONAL

Teaching scenario: choose objects on the table to build a small bridge



Published Astra test: block into bowl

Bridge task: a different, illustrative case

Available: two cans, a flat sheet,
a sponge and a short ruler.
Required: span a gap and carry a load.
First clarify the gap and load.

1 Propose roles and a sequence

Consider the cans as supports and the sheet as a deck.
Reject unsuitable candidates after physical checks.

2 Ask vision for measurements

Measure size, pose and available contact surfaces.
Check reach and clearance before planning a grasp.

3 Call executable robot tools

Use measured grasp poses and a checked motion plan.
Delegate a difficult manipulation to a learned skill if useful.

4 Verify, revise or ask for help

Inspect placement after each move. If stability or
load capacity is uncertain, stop and request validation.

An agent can propose what to do. Geometry, contact checks and verified control determine what can safely execute.

Method selection by situation

OPTIONAL

Use the simplest approach that meets the task and its acceptance tests

Situation	Main uncertainty	First approach to consider
Known red block, random position	Where is it? How is it oriented?	Calibrated vision + grasp planning. Add visual servoing if motion needs correction.
Unfamiliar rigid mug, clear pick-and-place goal	Identity and graspable 3D geometry	Detector/VLM for identity + depth + grasp planner. A new shape alone does not require VLA.
Crumpled towel, clear folding goal	Changing shape and contact	Task-adapted VLA or a specialised learned policy. Relevant data and recovery tests are essential.
“Build a bridge using things on this table”	Object roles, missing constraints and sequence	Astra-style reasoning + measurement + robot tools. Clarify requirements and validate the proposed plan.
Tight-fitting peg into a known hole	Small alignment error and contact forces	Calibrated vision + compliant/force control. An agent may select the routine, not certify precision.

Compare success rate, placement error, cycle time and data effort on representative trials before choosing.

Hybrid example: packing a mug with cloth

OPTIONAL

Teaching scenario: “Pack this mug safely in the box using the cloth”

STEP AND RESPONSIBLE METHOD	CONCRETE WORK AND REQUIRED RESULT
1 Interpret the request Astra-style task reasoning	Clarify protection and packing constraints. Propose a cloth liner, mug placement and handle protection before closing the box.
2 Locate and check fit Calibrated vision + geometry	Measure the mug, box opening and cloth location. Check grasp clearance and whether the mug fits. Return robot-frame targets.
3 Prepare the cloth liner VLA or a learned cloth skill	Use a relevant skill to grasp an edge and spread the cloth. Inspect coverage before allowing the mug-placement step.
4 Place the mug Motion planning + control	Move to the measured pose, approach slowly and release. Use contact sensing or compliance where the setup requires it.
5 Verify and choose recovery Perception + task supervisor	Re-measure a misplaced mug. Retry the cloth skill if coverage is poor. Ask for a new plan if the mug does not fit.

For a rigid insert and a fixed packing plan, conventional automation may be enough. Use VLA only where it helps.

Layered intelligence in one task

OPTIONAL

Packing the same mug: three different problems need three different responses

WHAT HAPPENS IN THE CASE	RESPONSE AND RESPONSIBLE LAYER
The mug shifts after detection →	**Fast local correction** Vision + control: re-measure the pose and realign. The packing goal and sequence stay the same.
Cloth bunches around the handle →	**Adapt a learned skill** VLA / learned policy: regrasp and spread the cloth. Report failure if recovery stalls or confidence is low.
The protected mug will not fit the box →	**Reconsider the plan** Task reasoning: choose a larger permitted box or ask. Measure again and check the revised plan before moving.

Throughout: motion control and protective limits stay active, even while the task planner is thinking.

Same robot: correct the motion, adapt the skill, or rethink the plan.
Layers describe functions. One model may implement several of them.


UNIVERSITY
OF WOLLONGONG
AUSTRALIA
Stand
for purpose
THANK YOU!